



UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

FACULTAD DE INGENIERÍA
CICLO 6

ESCUELA PROFESIONAL DE INGENIERÍA EN INFORMÁTICA Y
SISTEMAS

“GESTIÓN DE HILOS”

INFORME 04

SISTEMAS OPERATIVOS

PRESENTA

ALEX JEANPIER CHATA CHINO

2024-119012

23 de septiembre del 2026

1. Resumen

La presente práctica de laboratorio se centró en la implementación y análisis de la gestión de hilos utilizando la biblioteca POSIX Threads (pthread) en el lenguaje de programación C. El objetivo principal fue comprender el ciclo de vida de los hilos, desde su creación y ejecución concurrente hasta su terminación, evaluando el impacto de la sincronización mediante exclusión mutua (mutex). Se desarrollaron programas progresivos compilados con GCC para medir el rendimiento de ejecuciones con 1, 5 y 10 hilos. A través de este experimento, se buscó observar las diferencias entre la ejecución monohilo y multihilo, analizando particularmente las implicaciones del *overhead* (sobrecarga de gestión) y el comportamiento de la CPU frente a la concurrencia. Los resultados obtenidos permiten contrastar la teoría de la planificación de hilos y las condiciones de carrera con el comportamiento real del sistema operativo.

2. Introducción

En los sistemas operativos modernos, un hilo (*thread*) representa la unidad básica de utilización de la CPU, consistiendo en un contador de programa, un conjunto de registros y un espacio de pila. A diferencia de los procesos tradicionales que cuentan con espacios de memoria completamente aislados, los hilos que pertenecen a un mismo proceso comparten la sección de código, la sección de datos y los recursos del sistema operativo (Silberschatz, Galvin & Gagne, 2018). Esta característica permite que la comunicación y el cambio de contexto entre hilos sean significativamente más rápidos y eficientes que entre procesos.

La gestión adecuada de hilos es fundamental para desarrollar aplicaciones capaces de explotar el paralelismo en arquitecturas *multicore*. Sin embargo, la concurrencia introduce desafíos críticos, particularmente cuando múltiples hilos intentan acceder y modificar recursos compartidos simultáneamente. Para evitar anomalías como las condiciones de carrera, es imperativo implementar mecanismos de sincronización como los *mutex* (exclusión mutua) (Kerrisk, 2010). Esta práctica pretende demostrar, de manera empírica, la creación, ejecución y sincronización de hilos a nivel de usuario mapeados por el kernel de Linux, analizando cómo el grado de concurrencia afecta el rendimiento general de un programa.

3. Objetivos

3.1 Objetivo general

Implementar y analizar técnicamente la creación, gestión y sincronización de hilos concurrentes utilizando la biblioteca pthread en lenguaje C, evaluando su comportamiento y rendimiento en el sistema operativo.

3.2 Objetivos específicos

- Desarrollar rutinas de creación y unión de hilos mediante `pthread_create` y `pthread_join`.
- Implementar mecanismos de sincronización utilizando `pthread_mutex_t` para proteger secciones críticas.
- Comparar el rendimiento y los tiempos de ejecución entre un enfoque monohilo y arquitecturas multihilo (5 y 10 hilos).
- Identificar y explicar conceptualmente las condiciones de carrera y el impacto del *overhead* en la planificación del sistema operativo.

4. Metodología

4.1 Entorno de trabajo

El desarrollo de la práctica se llevó a cabo en un entorno con sistema operativo basado en Linux. Se utilizó el intérprete de comandos (terminal) para la interacción con el sistema, el editor de texto plano nano para la escritura del código fuente en lenguaje C, y el compilador GCC (*GNU Compiler Collection*) junto con la biblioteca estándar pthread.

4.2 Creación del programa

Se inició la práctica creando el archivo fuente mediante el editor de texto integrado en la terminal, ejecutando el comando `nano hilos.c`.

4.3 Implementación de creación de hilos

El código base se estructuró para declarar identificadores de tipo `pthread_t`. Se utilizó la función `pthread_create` para instanciar dos hilos independientes. A esta función se le enviaron como parámetros la referencia al identificador del hilo, los atributos por defecto (NULL), la rutina a ejecutar (`hilo_funcion`) y un argumento numérico convertido a puntero (`((void*)1` y `(void*)2`) para su identificación. Posteriormente, el hilo principal invocó `pthread_join` para suspender su propia ejecución hasta que los hilos secundarios terminaran.

Código inicial del programa de creación y gestión de hilos

```
GNU nano 8.7.1                                hilos.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void* hilo_funcion(void* arg)
{
    printf("Hola desde el hilo %ld\n", (long)arg);
    pthread_exit(NULL);
}

int main ()
{
    pthread_t hilo1, hilo2;

    // Creacion de los hilos
    pthread_create(&hilo1, NULL, hilo_funcion, (void*)1);
    pthread_create(&hilo2, NULL, hilo_funcion, (void*)2);

    // Esperar a que los hilos terminen su ejecucion
    pthread_join(hilo1, NULL);
    pthread_join(hilo2, NULL);

    printf("Finalizo la ejecucion del programa principal\n");
    return 0;
}
```

Figura 1. Estructura base del programa C con `pthread_create` y `pthread_join`.

4.4 Compilación

Para traducir el código fuente a un binario ejecutable, se utilizó el compilador GCC mediante el comando:

```
gcc -o hilos hilos.c -lpthread
```

La bandera `-lpthread` es estrictamente necesaria en este paso para enlazar dinámicamente la biblioteca POSIX Threads durante el proceso de compilación, resolviendo las referencias a funciones como `pthread_create`.

Compilación mediante GCC

```
unjbg@unjbg-VirtualBox:~/P03/practica4$ gcc -o hilos hilos.c -lpthread
```

Figura 2. Ejecución del comando de compilación en la terminal.

4.5 Ejecución inicial

El programa binario fue invocado desde la terminal ejecutando `./hilos`. La salida esperada consistió en la impresión en consola de los mensajes programados dentro de cada hilo secundario, seguidos del mensaje del hilo principal.

Primera ejecución del programa

```
unjbg@unjbg-VirtualBox:~/P03/practica4$ ./hilos
Hola desde el hilo 1
Hola desde el hilo 2
Finalizo la ejecucion del programa principal
unjbg@unjbg-VirtualBox:~/P03/practica4$
```

Figura 3. Salida estándar de la primera prueba del ejecutable.

4.6 Medición de tiempo

Se realizaron mediciones externas de tiempo utilizando el comando de terminal `time ./hilos`. Posteriormente, se integró la biblioteca `<time.h>` dentro del código para realizar una medición interna del tiempo de ejecución utilizando la función `clock()`. Es fundamental precisar que `clock()` mide el tiempo de CPU consumido por el proceso (la suma de los ciclos de CPU de todos los hilos). En un entorno multihilo sobre múltiples núcleos, este valor no representa el tiempo real transcurrido (*wall-clock time*), ya que los tiempos de CPU de hilos paralelos se acumulan.

Medición de tiempo de ejecución

```
Hola desde el hilo 1
Hola desde el hilo 2
Finalizado la ejecucion del programa principal

real    0m0,002s
user    0m0,000s
sys     0m0,002s
```

Figura 4. Salida de la ejecución mostrando los tiempos calculados.

4.7 Implementación de sincronización

Para proteger la función de impresión en consola (la sección crítica), se introdujo una variable global de tipo `pthread_mutex_t`. El mutex fue inicializado con `pthread_mutex_init`. Antes de que el hilo ejecutara el `printf`, se aplicó un bloqueo mediante `pthread_mutex_lock`, asegurando exclusión mutua. Tras la impresión, se liberó el recurso con `pthread_mutex_unlock`. Al finalizar el programa, se destruyó el mutex con `pthread_mutex_destroy`.

Implementación del mutex y sincronización



```
GNU nano 8.7.1                                hilos.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

pthread_mutex_t mutex;

void* hilo_funcion_con_mutex(void* arg)
{
    pthread_mutex_lock(&mutex);
    printf("Hola desde el hilo %ld\n", (long)arg);
    pthread_mutex_unlock(&mutex);
    pthread_exit(NULL);
}

int main ()
{
    pthread_t hilo1, hilo2;

    // Inicializar el mutex
    pthread_mutex_init(&mutex, NULL);

    // Creacion de los hilos
    pthread_create(&hilo1, NULL, hilo_funcion_con_mutex, (void*)1);
    pthread_create(&hilo2, NULL, hilo_funcion_con_mutex, (void*)2);

    // Esperar a que los hilos terminen su ejecucion
    pthread_join(hilo1, NULL);
    pthread_join(hilo2, NULL);

    // Destruir el mutex
    pthread_mutex_destroy(&mutex);

    printf("Finalizo la ejecucion del programa principal\n");
    return 0;
}
```

Figura 5. Código modificado para incluir exclusión mutua.

4.8 Prueba con diferentes cantidades de hilos

El código fue refactorizado dentro de una función ejecutar_con_hilos(int num_hilos) para permitir la instanciación dinámica mediante un arreglo de tipo pthread_t. Se probó el comportamiento sistemático del programa escalando la concurrencia a 1 hilo, 5 hilos y 10 hilos.

Pruebas de escalabilidad con 1, 5 y 10 hilos

```
GNU nano 8.7.1                                hilos.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

pthread_mutex_t mutex;

//Funcion del hilo
void* hilo_funcion(void* arg)
{
    pthread_mutex_lock(&mutex);
    printf("Hola desde el hilo %ld\n", (long)arg);
    pthread_mutex_unlock(&mutex);
    pthread_exit(NULL);
}

void ejecutar_con_hilos(int num_hilos)
{
    pthread_t hilos[num_hilos];

    // Inicializar el mutex
    pthread_mutex_init(&mutex, NULL);

    // Medicion de tiempo
    clock_t start, end;
    start = clock(); // Inicio del temporizador

    // Crear hilos
    for (long i = 0; i < num_hilos; i++)
    {
        pthread_create(&hilos[i], NULL, hilo_funcion, (void*)i);
    }

    // Esperar que los hilos terminen
    for (int i = 0; i < num_hilos; i++)
    {
        pthread_join(hilos[i], NULL);
    }

    end = clock();
}
```

```

    end = clock();

    // Destruir mutex
    pthread_mutex_destroy(&mutex);

    // Calcular y mostrar el tiempo de ejecucion
    double cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Tiempo total de ejecucion con %d hilos: %f segundos\n", num_hilos, cpu_time_used);
}

int main ()
{
    printf("Ejecutando con 1 hilo ...\n");
    ejecutar_con_hilos(1);
    printf("Ejecutando con 5 hilo ...\n");
    ejecutar_con_hilos(5);
    printf("Ejecutando con 10 hilo ...\n");
    ejecutar_con_hilos(10);

    printf("Finalizo la ejecucion del programa principal\n");
    return 0;
}

```

Figura 6 y 7. Código modificado para incluir la prueba con diferentes cantidades de hilos.

```

unjbg@unjbg-VirtualBox:~/P03/practica4$ gcc -o hilos hilos.c -lpthread

unjbg@unjbg-VirtualBox:~/P03/practica4$ ./hilos
Ejecutando con 1 hilo ...
Hola desde el hilo 0
Tiempo total de ejecucion con 1 hilos: 0.000597 segundos
Ejecutando con 5 hilo ...
Hola desde el hilo 0
Hola desde el hilo 1
Hola desde el hilo 3
Hola desde el hilo 2
Hola desde el hilo 4
Tiempo total de ejecucion con 5 hilos: 0.000753 segundos
Ejecutando con 10 hilo ...
Hola desde el hilo 0
Hola desde el hilo 1
Hola desde el hilo 2
Hola desde el hilo 5
Hola desde el hilo 7
Hola desde el hilo 3
Hola desde el hilo 9
Hola desde el hilo 8
Hola desde el hilo 4
Hola desde el hilo 6
Tiempo total de ejecucion con 10 hilos: 0.001432 segundos
Finalizo la ejecucion del programa principal

```

Figura 8. Resultados en terminal del bloque de pruebas de escalabilidad.

5. Resultados

Los datos obtenidos durante la experimentación para medir el impacto de la cantidad de hilos en el tiempo consumido se detallan en la siguiente tabla.

Número de hilos	Tiempo de CPU registrado (segundos)	Observación experimental
1	0.000597	Ejecución secuencial base. Representa el coste directo de creación y ejecución de una sola tarea.
5	0.000753	Se observa que los hilos tardan en ejecutar 0.000753 segundos, el cual muestra un ligero incremento de 0.000156 segundos.
10	0.001432	De acuerdo con los resultados registrados en la ejecución, el tiempo fue de 0.001432 segundos, evidenciando un incremento de casi el doble respecto a los 5 hilos.

Como se observa en la Figura 8, el programa logró ejecutarse de manera íntegra sin interrupciones abruptas. El uso del mutex garantizó que las líneas de texto se imprimieran de forma atómica.

6. Análisis de Resultados

6.1 Comparación entre programa monohilo y multihilo

Al comparar la ejecución monohilo con la multihilo, el tiempo de CPU reportado tendió a incrementar a medida que se añadían más hilos en lugar de disminuir (o en el mejor de los casos, permaneció estático con un ligero *overhead*). Esto ocurre porque el programa experimental está fuertemente ligado a E/S (operación `printf`) y ejecuta una carga de trabajo computacional minúscula. La gestión adecuada de hilos influye en el rendimiento general al permitir paralelismo real; sin embargo, cuando el coste de crear el hilo (*thread creation overhead*), planificarlo en el sistema operativo y coordinarlo con un mutex es mayor que la tarea misma, el rendimiento empeora (Kerrisk, 2010). Un programa multihilo solo será más rápido si la carga de trabajo paralela justifica el *overhead* estructural.

6.2 Funciones pthread utilizadas

- **pthread_create**: Inicializa y arranca la ejecución de un nuevo hilo. Es vital porque

instruye al kernel para asignar una nueva entidad de planificación sin duplicar el espacio de memoria del proceso padre.

- **pthread_join**: Actúa como un mecanismo de sincronización que bloquea al hilo invocador hasta que el hilo especificado termine. Es fundamental para evitar la generación de "hilos zombis" y garantizar que el proceso principal no finalice antes que sus tareas concurrentes.
- **pthread_mutex_lock** y **pthread_mutex_unlock**: Solicitan y liberan el bloqueo exclusivo de un recurso. Son el pilar de la sincronización POSIX, asegurando que solo un hilo a la vez pueda atravesar la sección crítica.

6.3 Aumento del número de hilos

De acuerdo con los resultados, al incrementar el número de hilos de 1 a 10, el tiempo reportado de 0.000597 segundos (1 hilo) aumentó drásticamente a 0.001432 segundos (10 hilos). Hubo un claro límite de escalabilidad. Esto ocurre debido a la "Contención" (*Contention*). Cuando múltiples hilos compiten por el mismo recurso protegido por un *mutex* (en este caso, el acceso a consola para imprimir), los hilos que no obtienen el bloqueo deben entrar en un estado de espera o suspensión (*sleep*). El sistema operativo invierte ciclos valiosos de CPU realizando *context switches* (cambios de contexto) para gestionar estos hilos en espera en lugar de ejecutar trabajo útil. Si el número de hilos supera el número de núcleos lógicos del procesador, la saturación y la degradación del rendimiento son más evidentes (Stallings, 2018).

6.4 Condiciones de carrera

En la versión inicial sin sincronización, el acceso concurrente a la función de impresión tenía el riesgo de provocar salidas intercaladas, dependiendo del búfer de la salida estándar (aunque la función `printf` suele ser *thread-safe* en implementaciones modernas de C de la librería GNU, su orden no es determinista). Las verdaderas condiciones de carrera se producen cuando múltiples hilos leen, modifican y escriben sobre una variable compartida sin sincronización, lo que resulta en corrupción de memoria o estados inconsistentes. Esto se previno exitosamente en la sección 5 del procedimiento implementando el *mutex*, el cual serializó artificialmente la sección crítica, sacrificando paralelismo puro para garantizar integridad (Tanenbaum & Bos, 2014).

7. Discusión Técnica

Los resultados demuestran la teoría detrás del modelo de hilos POSIX 1:1 nativo en Linux (NPTL - *Native POSIX Thread Library*). Cada hilo de usuario creado por `pthread_create` tiene un hilo del kernel correspondiente. La exclusión mutua observada ilustra el compromiso fundamental en el diseño de software concurrente: la seguridad frente al rendimiento. Al colocar un *mutex* alrededor del `printf`, el programa eliminó las condiciones de carrera lógicas, pero forzó

una ejecución secuencial dentro de un marco concurrente. Este fenómeno, sumado a que la función `clock()` en C mide el tiempo total consumido por el proceso en la CPU, explica por qué la concurrencia no siempre se traduce en un menor tiempo numérico en tareas tan cortas.

8. Conclusiones

- Se comprobó que la biblioteca POSIX Threads permite una derivación eficiente de flujos de control dentro de un mismo espacio de direcciones, logrando aislar lógicamente las tareas de manera sencilla mediante `pthread_create`.
- El escalamiento en la cantidad de hilos no representa automáticamente una mejora de rendimiento. Como se evidenció en la práctica empírica, al incrementar de 1 a 10 hilos en una tarea muy breve, el *overhead* y los tiempos de cambio de contexto superan cualquier beneficio de paralelización.
- La sincronización mediante `pthread_mutex_t` demostró ser indispensable y efectiva para proteger recursos compartidos, garantizando la consistencia en el estado del proceso al obligar al sistema a serializar el acceso a la sección crítica.

9. Referencias Bibliográficas

- Kerrisk, M. J. (2010). *The Linux programming interface: A Linux and UNIX system programming handbook*. No Starch Press.
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
- Stallings, W. (2018). *Operating systems: Internals and design principles* (9th ed.). Pearson.
- Tanenbaum, A. S., & Bos, H. (2014). *Modern Operating Systems* (4th ed.). Pearson.
- The GNU C Library. (n.d.). *POSIX Threads*. GNU Project.
https://www.gnu.org/software/libc/manual/html_node/POSIX-Threads.html